Reviewer Pack — Minimal Rank of Tropicalized Graph Laplacians vs BP_ReadTwic…

Entry 5f775fefd0de · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 21:53:15 UTC

Minimal Rank of Tropicalized Graph Laplacians vs BP_ReadTwice Circuit Depth

Entry ID: `5f775fefd0de` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 21:53:15 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Laplacian matrices) **Field B** (complexity object): Complexity Theory: Branching Program Complexity (BP_ReadTwice)

Statement:

> {'specific': 'For every graph G with n vertices, let L_G be the Laplacian matrix associated with G. Define ρ(L_G) as the rank of the tropicalized Laplacian, which is obtained by replacing each entry of L_G with its nonnegative part and then converting to the tropical semiring. Then for all graphs G, ρ(L_G) = O(log n) but ρ(L_IP_2) = Ω(n^2).', 'counterexample': 'Find a graph G with a Laplacian matrix L_G such that ρ(L_G) ≤ c·log n and ρ(L_IP_2) < d·n^2 for some constants c, d.'}

Rationale (proposer's reasoning):

> {'explanation': 'The tropicalization of Laplacian matrices has been studied in the context of network analysis, but its application to complexity theory is novel. The conjecture leverages the structure of tropical semirings to provide a distinguisher for read-twice BP complexity, which could shed light on the separation between read-once and read-twice models.', 'justification': 'Laplacian matrices capture the connectivity of graphs, and their tropicalization offers a geometric representation that may expose hidden structures not evident in classical matrix operations.'}

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2b17eca5fa1e0416`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a graph G with n vertices, if ρ(L_G) is within O(log n) of log n and ρ(L_IP_2) is at least Ω(n^2), then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry laplacian matrices AND BP_ReadTwice circuit depth` - `graph Laplacian matrix rank tropicalization AND complexity theory BP_ReadTwice` - `log n bound on tropicalized Laplacian rank AND Ω(n^2) for IP_2 graph`

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1806.00072v2] On graph Laplacian eigenvectors with components in {-1,0,1} - [http://arxiv.org/abs/1301.6896v1] Laplacians on periodic discrete graphs - [http://arxiv.org/abs/1007.2472v3] Laplacian spectral characterization of some graph products - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        if matrix[i][i] == 0:
            # Find a non-zero pivot below and swap rows
            for j in range(i + 1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            else:
                # No non-zero pivot found, skip this column
                continue

        factor = Fraction(1, matrix[i][i])
        for j in range(n):
            matrix[i][j] *= factor
```

```
            for k in range(i + 1, n):
                factor = matrix[k][i]
                for j in range(n):
                    matrix[k][j] -= factor * matrix[i][j]

        rank = sum(1 for row in matrix if any(row))
        return rank

    def tropical_rank(matrix):
        n = len(matrix)
        T = [[max(a, b) for a, b in zip(row, col)] for row, col in zip(*matrix)]
        return gaussian_elimination(T)

    def laplacian_matrix(G):
        n = len(G)
        L = [[0] * n for _ in range(n)]
        for i in range(n):
            degree = sum(1 for j in range(n) if G[i][j])
            L[i][i] = -degree
            for j in range(i + 1, n):
                L[i][j] = L[j][i] = G[i][j]
        return L

    def run_trial(seed: int) -> dict:
        random.seed(seed)

        if not sys.argv[1:]:
            seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
        else:
            seeds = [int(s) for s in sys.argv[1:]]

        results = []
        for n in {5, 10, 15, 20, 30, 40}:
            for _ in range(5):
                G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
                L_G = laplacian_matrix(G)
                rho_L_G = tropical_rank(L_G)

                L_IP_2 = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
                rho_L_IP_2 = gaussian_elimination(L_IP_2)

                results.append({
                    "metric_name": "rho",
                    "metric_value": rho_L_G,
                    "instances_tested": 1,
                    "conjecture_holds": rho_L_G <= math.log(n) and rho_L_IP_2 >= n**2,
                    "counterexample": ""
                })

        mean_metric = sum(result["metric_value"] for result in results) / len(results)
        std_metric = (sum((result["metric_value"] - mean_metric)**2 for result in results) / len(resul
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

        return {
            "seed": seed,
            "mean_metric": mean_metric,
```

```
        "std_metric": std_metric,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    import sys
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac35c36c.py", line 97, in <module>
    for seed in seeds:
                ^^^^^
NameError: name 'seeds' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture's support conditions are met. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10635
2	propose	ollama_remote	glm4:latest	0	0	12261

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	propose	ollama_remote	glm4:latest	0	0	12498
4	preregistration	ollama_remote	glm4:latest	0	0	5843
5	novelty	ollama_remote	glm4:latest	0	0	4797
6	novelty	ollama_remote	glm4:latest	0	0	5831
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23169
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11205
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13413
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10659
11	judge	ollama_remote	glm4:latest	0	0	8204

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118516 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/5f775fefd0de.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5f775fefd0de.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5f775fefd0de.tar.gz` (if generated)